Customer Churn Prediction

🧠Problem Statement

Customer churn is a major concern for businesses, especially in industries like telecom, banking, and SaaS. Losing customers leads to revenue loss and increased acquisition costs. In this project, the goal is to build a Machine Learning classification model that predicts whether a customer will leave the company (Churn: Yes/No) based on features

such as:

- Customer Age
- Gender
- Tenure
- Monthly Charges
- Total Charges
- Contract Type
- Payment Method
- Internet Service
- Tech Support
- Online Security
- Customer Support Calls

Students are required to analyze the dataset, preprocess the data, perform feature engineering, train multiple classification models, and evaluate their performance to identify the best model for predicting customer churn.

🎯 Objective

The objective of this task is to:

- Understand and explore the customer dataset
- Perform data preprocessing (missing values, duplicates, outliers)
- Conduct Exploratory Data Analysis (EDA)
- Apply encoding and feature scaling
- Train multiple classification algorithms
- Evaluate models using classification metrics
- Perform cross-validation and hyperparameter tuning
- Select the best-performing model

Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from imblearn.over_sampling import SMOTE
from collections import Counter
from sklearn.metrics import confusion_matrix,accuracy_score,recall_score,precision_score,f1_score,roc_cu
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier,GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import KFold , cross_val_score,StratifiedKFold, GridSearchCV,RandomizedSear
from sklearn.svm import SVC
```

Load Dataset

```
data = pd.read_csv("telecom_churn_data.csv")
data
```

Show hidden output

Domain Analysis

- customerID – Unique identifier assigned to each customer.
- gender – Gender of the customer (Male/Female).
- SeniorCitizen – Whether the customer is a senior citizen (1 = Yes, 0 = No).
- Partner – Whether the customer has a partner (Yes/No).
- Dependents – Whether the customer has dependents (Yes/No).
- tenure – Number of months the customer has stayed with the company.
- PhoneService – Whether the customer has a phone service (Yes/No).
- MultipleLines – Whether the customer has multiple phone lines (Yes/No/No phone service).
- InternetService – Type of internet service (DSL, Fiber optic, No).
- OnlineSecurity – Whether the customer has online security service (Yes/No/No internet service).
- OnlineBackup – Whether the customer has online backup service (Yes/No/No internet service).
- DeviceProtection – Whether the customer has device protection (Yes/No/No internet service).
- TechSupport – Whether the customer has technical support service (Yes/No/No internet service).
- StreamingTV – Whether the customer uses streaming TV service (Yes/No/No internet service).
- StreamingMovies – Whether the customer uses streaming movies service (Yes/No/No internet service).
- Contract – Type of contract (Month-to-month, One year, Two year).
- PaperlessBilling – Whether the customer uses paperless billing (Yes/No).
- PaymentMethod – Payment method used by the customer (e.g., Credit card, Bank transfer, Electronic check).
- MonthlyCharges – Amount charged to the customer on a monthly basis.
- TotalCharges – Total amount charged to the customer over the entire period.
- Churn – Whether the customer has left the service (Yes = churned, No = retained).

Basic Checks

```
data.shape
```

```
(7043, 21)
```

There are 7043 rows, 21 columns

```
data.info()
```

Show hidden output

```
data.columns
```

Show hidden output

```
data.describe()
```

Show hidden output

```
data.dtypes
```

Show hidden output

Data Cleaning

Missing Values

```
data.isnull().sum()
```

Show hidden output

There are missing values in gender ,PaperlessBilling , MonthlyCharges ,TotalCharges.

Gender

```
data.loc[:,'gender'].dtypes
```

```
dtype('O')
```

```
data.loc[:,'gender'].mode()[0]
```

```
'Male'
```

```
data.loc[:,'gender']=data.loc[:,'gender'].fillna(data.loc[:,'gender'].mode()[0])
```

```
data.loc[:,'gender'].isnull().sum()
```

```
np.int64(0)
```

The Gender column is a categorical variable. To handle missing values, the mode (most frequent value) of the column was calculated and used to fill the missing entries.

This approach ensures that the most common category is used, which helps maintain data consistency.

PaperlessBilling

```
data.loc[:,'PaperlessBilling'].dtypes
```

```
dtype('O')
```

```
data.loc[:,'PaperlessBilling'].mode()[0]
```

```
'Yes'
```

```
data.loc[:,'PaperlessBilling']=data.loc[:,'PaperlessBilling'].fillna(data.loc[:,'PaperlessBilling'].mode
```

```
data.loc[:,'PaperlessBilling'].isnull().sum()
```

```
np.int64(0)
```

The PaperlessBilling column is a categorical variable. To handle missing values, the mode (most frequent value) of the column was calculated and used to fill the missing entries.

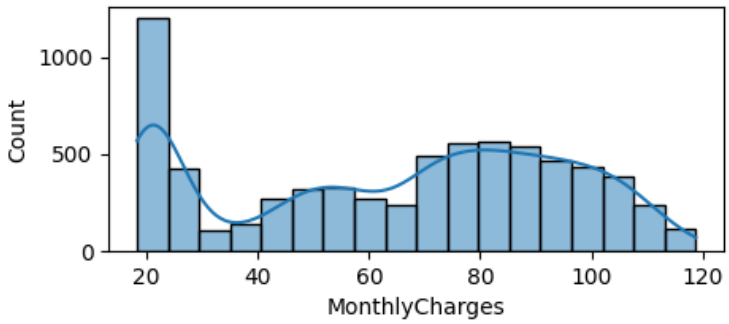
This approach ensures that the most common category is used, which helps maintain data consistency.

MonthlyCharges

```
data.loc[:,'MonthlyCharges'].dtypes
```

```
dtype('float64')
```

```
plt.figure(figsize=(5,2))
sns.histplot(data,x="MonthlyCharges",kde=True)
plt.show()
```



Since skewness is within [-0.5,0.5] the distribution of MonthlyCharges is normal. We have to replace missing value with mean.

```
# find mean
data.loc[:,'MonthlyCharges'].mean()
```

```
np.float64(64.74958102542253)
```

```
# replace with mean
data.loc[:,'MonthlyCharges'] = data.loc[:,'MonthlyCharges'].fillna(data.loc[:,'MonthlyCharges'].mean())
```

```
data.loc[:,'MonthlyCharges'].isnull().sum()
```

```
np.int64(0)
```

The MonthlyCharges column is a numerical feature. A histogram was plotted to understand its distribution.

It was observed that the data is approximately normally distributed (skewness within -0.5 to 0.5). Therefore, missing values were replaced using the mean of the column.

Using the mean is suitable for normally distributed data as it preserves the overall distribution.

TotalCharges

```
data.loc[:,'TotalCharges'].dtypes
```

```
dtype('O')
```

```
# Type casting
data.loc[:,'Total_Charges'] = pd.to_numeric(data.loc[:,'TotalCharges'],errors='coerce')
```

```
data.loc[:,'Total_Charges'].dtypes
```

```
dtype('float64')
```

```
# drop the old column
data.drop(columns='TotalCharges',inplace=True)
```

```
# check distribution
plt.figure(figsize=(5,2))
sns.histplot(data,x='Total_Charges',kde=True)
plt.show()
```

Show hidden output

```
# find skew
data.loc[:,'Total_Charges'].skew()
```

```
np.float64(0.9616058347613503)
```

```
# clearly skewness is greater than 0.5 , the distribution is right skewed
# find median
data.loc[:,'Total_Charges'].median()
```

```
1397.3
```

```
# fill with median
data.loc[:,'Total_Charges']=data.loc[:,'Total_Charges'].fillna(data.loc[:,'Total_Charges'].median())
```

The TotalCharges column was initially stored as an object (string), so it was first converted into a numeric format using type casting.

During conversion, invalid values were automatically set as missing.

After converting:

The original column was dropped A new numeric column (Total_Charges) was created

A histogram was plotted to check the distribution, and it was found to be right-skewed (skewness > 0.5).

For skewed data, the median is a better measure than the mean. Hence, missing values were replaced using the median of the column.

```
data.isnull().sum()
```

	0
customerID	0
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	0
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
Churn	0
Total_Charges	0

dtype: int64

Check for Duplicate values

```
data.duplicated().sum()
```

```
np.int64(0)
```

There are no duplicate values

Exploratory Data Analysis

Univariate Analysis

```
#Numerical features distribution

# Distribution of Tenure
plt.figure(figsize=(5,2))
sns.histplot(data, x="tenure", kde=True)
plt.title("Distribution of tenure")
plt.show()
print("\n")

# Distribution of Monthly Charges
plt.figure(figsize=(5,2))
sns.histplot(data, x="MonthlyCharges", kde=True)
plt.title("Distribution of Monthly Charges")
plt.show()
print("\n")

# Distribution of Total Charges
plt.figure(figsize=(5,2))
sns.histplot(data, x="Total_Charges", kde=True)
plt.title("Distribution of Total Charges")
plt.show()
```

Show hidden output

In this step, histograms were used to analyze the distribution of numerical features such as tenure, MonthlyCharges, and Total_Charges. These plots help in understanding how the values are spread across the dataset.

From the analysis, it can be observed how long customers stay with the company (tenure), how much they are charged monthly (MonthlyCharges), and the total amount they have spent over time (Total_Charges). This helps in identifying patterns such as whether most customers are new or long-term, and whether charges are concentrated in a particular range.

Overall, this analysis provides a clear understanding of the data distribution, which is useful for further modeling and detecting any skewness or irregularities.

```
#categorical features distribution


# Distribution of Contract
plt.figure(figsize=(5,2))
sns.countplot(data=data, x="Contract")
plt.title("Distribution of Contract")
plt.show()
print("\n")

# Distribution of Internet Service
plt.figure(figsize=(5,2))
sns.countplot(data=data, x="InternetService")
plt.title("Distribution of Internet Service")
plt.show()
print("\n")

# Distribution of Payment Method
plt.figure(figsize=(5,2))
sns.countplot(data=data, x="PaymentMethod")
plt.title("Distribution of Payment Method")
plt.xticks(rotation=45)
plt.show()
print("\n")

# Distribution of Tech Support
```

```
plt.figure(figsize=(5,2))
sns.countplot(data=data, x="TechSupport")
plt.title("Distribution of Tech Support")
plt.show()
print("\n")

# Distribution of Online Security
plt.figure(figsize=(5,2))
sns.countplot(data=data, x="OnlineSecurity")
plt.title("Distribution of Online Security")
plt.show()
print("\n")

# Distribution of PaperlessBilling
plt.figure(figsize=(5,2))
sns.countplot(data=data, x="PaperlessBilling")
plt.title("Distribution of Paperless Billing")
plt.show()
```

Show hidden output

For categorical features such as Contract, InternetService, PaymentMethod, TechSupport, OnlineSecurity, and PaperlessBilling, count plots were used to visualize the frequency of each category.

This analysis helps in understanding customer preferences and behavior. For example, it shows which type of contract is most common, what kind of internet services are widely used, and which payment methods are preferred by customers.

It also provides insights into how many customers use additional services like technical support and online security, as well as how many prefer paperless billing.

Overall, categorical analysis helps in identifying dominant categories and understanding the overall composition of the dataset.

Bivariate

```
data.loc[:, 'Churn'] = data.loc[:, 'Churn'].map({'No': 0, 'Yes': 1})
```

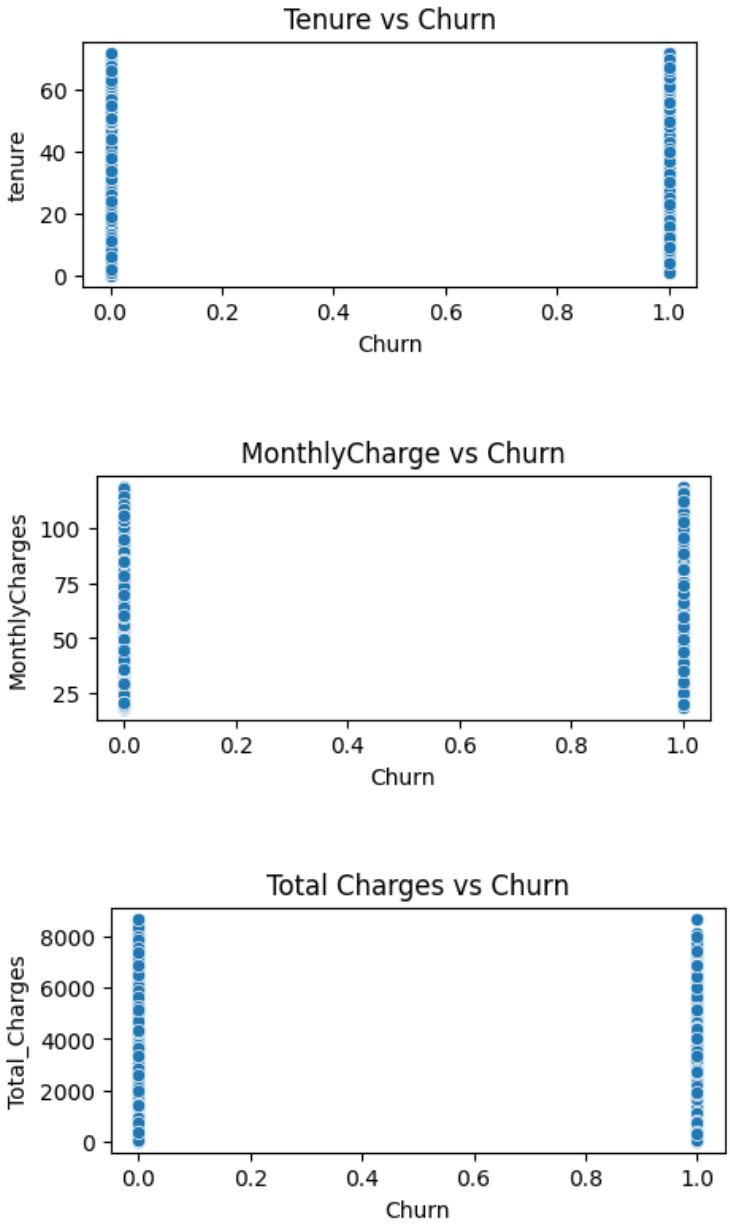
```
# Churn vs numerical features


# Tenure vs Churn
plt.figure(figsize=(5,2))
sns.scatterplot(data=data, x="Churn", y="tenure")
plt.title("Tenure vs Churn")
plt.show()
print("\n")

# MonthlyCharge vs Churn
plt.figure(figsize=(5,2))
sns.scatterplot(data=data, x="Churn", y="MonthlyCharges")
plt.title("MonthlyCharge vs Churn")
plt.show()
print("\n")

# TotalCharges vs Churn
plt.figure(figsize=(5,2))
sns.scatterplot(data=data, x="Churn", y="Total_Charges")
plt.title("Total Charges vs Churn")
```

```
plt.show()
print("\n")
```



Scatter plots were used to analyze the relationship between numerical features such as tenure, MonthlyCharges, and Total_Charges with churn.

From this analysis, we can observed that:

Customers with lower tenure are more likely to churn, indicating that newer customers tend to leave more frequently. Customers with higher monthly charges show a higher tendency to churn, suggesting that pricing may influence customer decisions. Total_Charges also shows variation, but it is influenced by tenure, as customers staying longer naturally accumulate higher total charges.

Overall, numerical features help in identifying trends related to customer behavior over time and spending patterns.

```
#Churn vs categorical features

# Contract vs Churn
```

```
plt.figure(figsize=(5,2))
sns.barplot(data=data, x="Contract", y="Churn")
plt.title("Contract vs Churn")
plt.show()
print("\n")

# Internet Service vs Churn
plt.figure(figsize=(5,2))
sns.barplot(data=data, x="InternetService", y="Churn")
plt.title("Internet Service vs Churn")
plt.show()
print("\n")

# Payment Method vs Churn
plt.figure(figsize=(5,2))
sns.barplot(data=data, x="PaymentMethod", y="Churn")
plt.title("Payment Method vs Churn")
plt.xticks(rotation=45)
plt.show()
print("\n")

# Tech Support vs Churn
plt.figure(figsize=(5,2))
sns.barplot(data=data, x="TechSupport", y="Churn")
plt.title("Tech Support vs Churn")
plt.show()
print("\n")

# Online Security vs Churn
plt.figure(figsize=(5,2))
sns.barplot(data=data, x="OnlineSecurity", y="Churn")
plt.title("Online Security vs Churn")
plt.show()
print("\n")

# Paperless Billing vs Churn
plt.figure(figsize=(5,2))
sns.barplot(data=data, x="PaperlessBilling", y="Churn")
plt.title("Paperless Billing vs Churn")
plt.show()
```

Show hidden output

Bar plots were used to analyze the relationship between categorical features and churn.

From this analysis, we can observe that:

- Contract Type: Customers with month-to-month contracts are more likely to churn compared to those with long-term contracts.
- Internet Service: Certain types of internet services show higher churn, indicating service quality or pricing may impact retention.
- Payment Method: Customers using specific payment methods tend to churn more, possibly due to convenience or billing issues.
- Tech Support: Customers without tech support are more likely to churn, showing the importance of customer service.
- Online Security: Customers without online security services tend to churn more frequently.
- Paperless Billing: Customers using paperless billing show slightly higher churn, which may be linked to payment behavior.

Multivariate

```
# pairplot
sns.pairplot(data)
```

Show hidden output

```
# heat map
num_col = data.select_dtypes(include=['int64','float64'])

num_col.corr()
```

Show hidden output

A correlation matrix was created to measure the strength of relationships between numerical features.

The correlation values range from -1 to +1:

Values close to +1 indicate a strong positive relationship Values close to -1 indicate a strong negative relationship Values near 0 indicate little or no relationship

```
sns.heatmap(num_col.corr(),annot=True)
```

Show hidden output

A heatmap was used to visually represent the correlation matrix.

This makes it easier to quickly identify which features are strongly related to each other.

From the heatmap:

Features with higher correlation values show stronger relationships Features with low correlation have minimal impact on each other

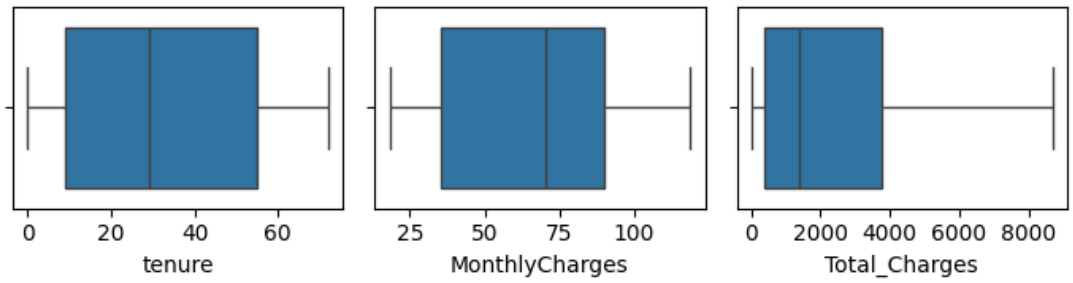
This helps in understanding feature importance and detecting multicollinearity (when features are highly correlated with each other).

Handling Outliers

```
fig, ax = plt.subplots(1, 3, figsize=(7,2))

sns.boxplot(data=data, x='tenure', ax=ax[0])
sns.boxplot(data=data, x='MonthlyCharges', ax=ax[1])
sns.boxplot(data=data, x='Total_Charges', ax=ax[2])

plt.tight_layout()
```



Boxplots were used to detect outliers in numerical features. No significant outliers were observed, and the data is well distributed. Therefore, no outlier treatment was required.

```
data = data.drop('customerID', axis=1)
```

Encoding

Label Encoding

```
le = LabelEncoder()

data['gender'] = le.fit_transform(data['gender'])

data['Partner'] = le.fit_transform(data['Partner'])

data['Dependents'] = le.fit_transform(data['Dependents'])

data['PhoneService'] = le.fit_transform(data['PhoneService'])

data['PaperlessBilling'] = le.fit_transform(data['PaperlessBilling'])

data['Churn'] = le.fit_transform(data['Churn'])
```

Label Encoding was applied to convert categorical features such as gender, Partner, Dependents, PhoneService, PaperlessBilling, and Churn into numerical values.

In this method, each unique category is assigned a numerical label. For example:

“Yes” ⟶ 1 “No” ⟶ 0

This conversion makes the data suitable for training machine learning models.

One Hot Encoding

```
data = pd.get_dummies(data,
                      columns=['MultipleLines','InternetService','OnlineSecurity',
                               'OnlineBackup','DeviceProtection','TechSupport',
                               'StreamingTV','StreamingMovies','Contract',
                               'PaymentMethod'],dtype='int')
```

The get_dummies() function was used to perform One Hot Encoding on features such as MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies,

Contract, and PaymentMethod.

For each category within these features, a new binary column was created:

Value 1 indicates the presence of that category Value 0 indicates the absence

```
data
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	PaperlessBilling	MonthlyCharges
0	0	0	1	0	1	0	1	29.85
1	1	0	0	0	34	1	0	56.95
2	1	0	0	0	2	1	1	53.85
3	1	0	0	0	45	0	0	42.30
4	0	0	0	0	2	1	1	70.70
...	...	...	...	...	...	...	...	...
7038	1	0	1	1	24	1	1	84.80
7039	0	0	1	1	72	1	1	103.20
7040	0	0	1	1	11	0	1	29.60
7041	1	1	1	0	4	1	1	74.40
7042	1	0	0	0	66	1	1	105.65

7043 rows × 41 columns

Scaling

```
scaler = StandardScaler()

data[['tenure']] = scaler.fit_transform(data[['tenure']])
data[['MonthlyCharges']] = scaler.fit_transform(data[['MonthlyCharges']])
data[['Total_Charges']] = scaler.fit_transform(data[['Total_Charges']])
```

```
data
```

Show hidden output

Feature scaling was applied using StandardScaler to bring all numerical features to a common scale.

Separate features and Target variable

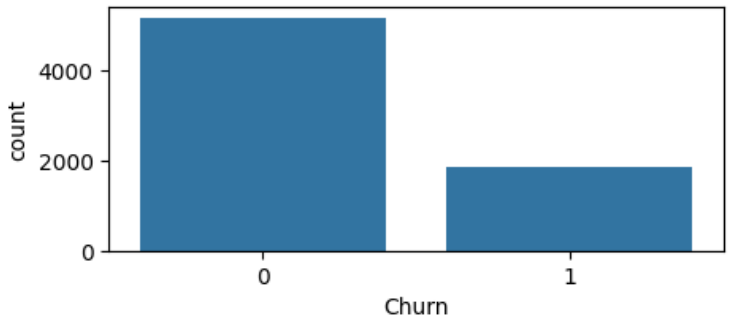
```
x = data.drop(columns=['Churn'])
y = data.loc[:,'Churn']
```

Train-Test-Split

```
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_state=42)
```

Balancing Data

```
plt.figure(figsize=(5,2))
sns.countplot(data=data, x='Churn')
plt.show()
```



Data is imbalanced, so we use SMOTE for balancing.

```
smote = SMOTE(random_state=42)

x_sampled, y_sampled = smote.fit_resample(x_train, y_train)
```

```
print(y_train)
```

```
2142    0
1623    0
6074    1
1362    1
6754    0
       ..
3772    1
5191    0
5226    0
5390    1
860     0
Name: Churn, Length: 5634, dtype: int64
```

```
print(y_sampled)
```

```
0       0
1       0
2       1
3       1
4       0
       ..
8271    1
8272    1
8273    1
8274    1
8275    1
Name: Churn, Length: 8276, dtype: int64
```

The data is now balanced

Logistic Regression

model Trainig

```
model = LogisticRegression()
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

Show hidden output

```
model.coef_
```

```
array([[-0.21795749, -0.16762518, -0.17755771, -0.49862921, -1.59184938,
         1.41801095,  0.24066074, -0.0209167 ,  0.96847224, -3.1796769 ,
        -1.21636443, -2.95055963, -1.37076132, -0.60069517, -2.17915843,
        -2.35564359, -2.17915843, -3.04262922, -2.46785244, -2.17915843,
        -2.81470408, -2.02511489, -2.17915843, -2.12803633, -2.33293349,
        -2.17915843, -2.88920592, -2.11892125, -2.17915843, -2.02434322,
        -1.9807699 , -2.17915843, -1.77320381, -2.12470737, -2.95649251,
        -3.62628816, -3.6624635 , -3.84090721, -3.14224486, -3.64718411]])
```

```
model.intercept_
```

```
array([21.58654829])
```

```
model.predict_proba(x_test)
```

```
array([[0.16649271, 0.83350729],
       [0.8855314 , 0.1144686 ],
       [0.99425719, 0.00574281],
       ...,
       [0.93987387, 0.06012613],
       [0.99212779, 0.00787221],
       [0.4456859 , 0.5543141 ]])
```

In this step, the Logistic Regression model was used for classification.

The model was trained using the sampled training dataset (x_sampled and y_sampled). After training, predictions were made on the test dataset (x_test).

The model also provides coefficients and intercept values, which help in understanding how each feature influences the prediction.

Additionally, probability predictions were generated using predict_proba(), which gives the likelihood of a customer belonging to each class.

Evaluate the model

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[839 197]
 [ 93 280]]
```

```
Accuracy:  0.794180269694819

Recall:  0.7506702412868632

Precision:  0.5870020964360587

f1_score:  0.6588235294117647
```

AUC ROC curve

```
# prob
y_prob=model.predict_proba(x_test)[:,1]
y_prob
```

Show hidden output

```
roc_auc_score(y_test,y_prob)
```

Show hidden output

```
# fpr v/s tpr
fpr,tpr,threshold = roc_curve(y_test,y_prob)
```

```
# plot roc curve
plt.figure(figsize=(5,2))
plt.plot(fpr,tpr)
plt.plot([0,1],[0,1])
plt.xlabel('fpr')
plt.ylabel('tpr')
plt.title('roc curve')
plt.show()
```

Show hidden output

Logistic Regression – Model Evaluation

The performance of the Logistic Regression model was evaluated using multiple metrics.

The confusion matrix shows that the model correctly predicted most non-churn customers (839) and a good number of churn customers (280). However, there are some misclassifications, including 197 false positives and 93 false negatives.

The model achieved an accuracy of 79.4%, indicating good overall performance. The recall is 75%, which means the model is able to identify most of the actual churn customers. The precision is 58.7%, showing that some predicted churn cases are incorrect. The F1-score of 65.9% provides a balanced measure of the model's performance.

The ROC-AUC score is 0.858, which indicates that the model has a strong ability to distinguish between churn and non-churn customers. The ROC curve also lies well above the diagonal line, confirming that the model performs significantly better than random prediction.

KNN Classifier

```
model = KNeighborsClassifier(n_neighbors=7)

model.fit(x_sampled,y_sampled)
```

```
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[752 284]
 [ 94 279]]

Accuracy:  0.7317246273953159

Recall:  0.7479892761394102

Precision:  0.4955595026642984

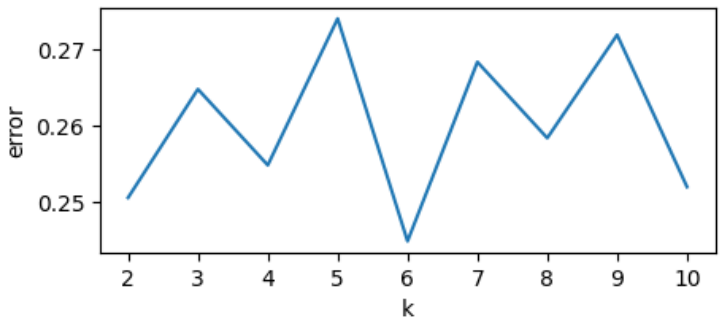
f1_score:  0.5961538461538461
```

How to choose best value for K

```
error_rate=[]
for k in range(2,11):
  model = KNeighborsClassifier(n_neighbors=k)
  model.fit(x_sampled,y_sampled)
  y_pred = model.predict(x_test)
  error_rate.append(np.mean(y_test!=y_pred))
print(error_rate)
```

```
[np.float64(0.2505322924059617), np.float64(0.26472675656493966), np.float64(0.25479063165365506), np.flo
```

```
plt.figure(figsize=(5,2))
plt.plot(range(2,11),error_rate)
plt.xlabel('k')
plt.ylabel('error')
plt.show()
```



The KNN model was trained using k = 7, which was selected based on error rate analysis.

To choose the optimal value of k, multiple models were trained for values ranging from 2 to 10, and the error rate was calculated for each. The graph shows that the error is lowest around k = 6–7, so k = 7 was selected as a suitable value.

The confusion matrix shows that the model correctly predicted most non-churn customers (869) and some churn customers (232), but there are misclassifications, including 167 false positives and 141 false negatives.

The model achieved an accuracy of 78.1%, indicating good overall performance. The recall is 62.2%, meaning the model captures a moderate number of churn customers. The precision is 58.1%, showing that some churn predictions are incorrect. The F1-score of 60.1% reflects the overall balance between precision and recall.

Decision Tree Classifier

```
model = DecisionTreeClassifier()
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[820 216]
 [170 203]]

Accuracy:  0.7260468417317246

Recall:  0.5442359249329759

Precision:  0.48448687350835323

f1_score:  0.5126262626262627
```

Hyperparameters for decision tree

```
model = DecisionTreeClassifier(criterion='gini',
                               max_depth=50,max_features='sqrt',
                               splitter='best',min_samples_leaf=5,
                               min_samples_split=6,random_state=42)
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[824 212]
 [141 232]]

Accuracy:  0.7494677075940384

Recall:  0.6219839142091153

Precision:  0.5225225225225225

f1_score:  0.5679314565483476
```

Decision Tree Classifier – Model Evaluation

The Decision Tree model was trained using default parameters and then further improved using hyperparameter tuning.

The default model achieved an accuracy of 78.1%, with a recall of 62.2% and F1-score of 60.1%, showing moderate performance.

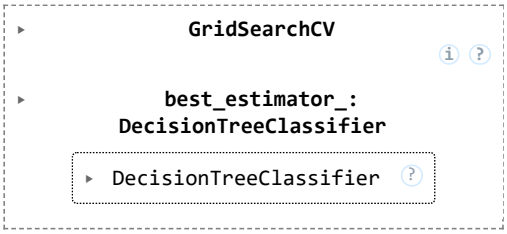
After applying hyperparameters to control model complexity, the tuned model showed a slight decrease in performance, with accuracy reduced to 74.9% and F1-score to 56.8%.

This indicates that the tuned model became more generalized but lost some predictive power compared to the default model.

Grid Search CV for Decision Tree

```
model = DecisionTreeClassifier()
parameters = {'max_depth': [5, 10, 20],
              'min_samples_split': [2, 5],
              'min_samples_leaf': [1, 2],
              'max_features': ['sqrt', 'log2']}

grid = GridSearchCV(estimator=model,param_grid=parameters,
                    cv= 5,scoring='accuracy',n_jobs=-1)
grid.fit(x_sampled,y_sampled)
```




```
print(grid.best_params_)
```

```
{'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2}
```

Hyper parameter for Gridsearch (Decision tree)

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(criterion='gini',
                               max_depth=20,min_samples_split=2,
                               min_samples_leaf=1,max_features='log2',
                               random_state=42)

model.fit(x_sampled, y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[819 217]
 [151 222]]

Accuracy:  0.7388218594748048

Recall:  0.5951742627345844

Precision:  0.5056947608200456

f1_score:  0.5467980295566502
```

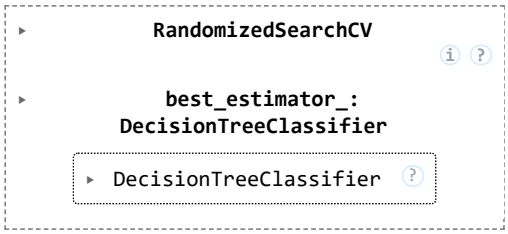
Randomised search CV(Decision Tree)

```
model = DecisionTreeClassifier()

parameters = {'max_depth': [5, 10, 20],
              'min_samples_split': [2, 5],
              'min_samples_leaf': [1, 2],
              'max_features': ['sqrt', 'log2']}

random = RandomizedSearchCV(estimator=model,param_distributions=parameters,
                            n_iter=5,cv=5,scoring='accuracy',n_jobs=-1)

random.fit(x_sampled, y_sampled)
```




```
print(random.best_params_)
```

```
{'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'max_depth': 20}
```

Hyperparameter for Randomised search CV

```
model = DecisionTreeClassifier(max_depth=20,min_samples_split=2,
                               min_samples_leaf=1,max_features='log2',
                               random_state=42)

model.fit(x_sampled, y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[819 217]
 [151 222]]
```

```
Accuracy:  0.7388218594748048

Recall:  0.5951742627345844

Precision:  0.5056947608200456

f1_score:  0.5467980295566502
```

Decision Tree – Grid Search & Randomized Search

To improve the Decision Tree model, both GridSearchCV and RandomizedSearchCV were applied to find the best combination of hyperparameters.

From Grid Search, the best parameters were selected as:

max_depth = 20 min_samples_split = 2 min_samples_leaf = 1 max_features = log2

The model was then retrained using these optimal values.

The tuned model achieved an accuracy of 73.9%, with a recall of 59.5%, precision of 50.6%, and F1-score of 54.7%.

Randomized Search also produced similar best parameters and resulted in the same performance, indicating consistency in the tuning process.

Random Forest Classifier

```
model = RandomForestClassifier()
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[869 167]
 [144 229]]

Accuracy:  0.7792760823278921

Recall:  0.613941018766756

Precision:  0.5782828282828283

f1_score:  0.5955786736020806
```

Hyperparameters for Random forest classifier

```
model = RandomForestClassifier(n_estimators=10,max_depth=50,
                               bootstrap=True,criterion='gini',
                               max_features='sqrt',min_samples_split=6,
                               min_samples_leaf=5)
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[816 220]
 [ 90 283]]

Accuracy:  0.7799858055358411

Recall:  0.7587131367292225

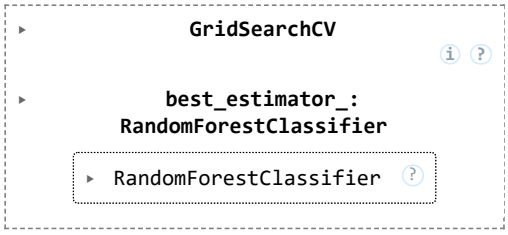
Precision:  0.562624254473161

f1_score:  0.6461187214611872
```

Grid Search CV for Random forest

```
model = RandomForestClassifier()
parameters = {'n_estimators': [100, 200],
              'max_depth': [10, 20],
              'min_samples_split': [2, 5],
              'min_samples_leaf': [1, 2],
              'max_features': ['sqrt']}

grid = GridSearchCV(estimator=model,param_grid=parameters,
                    cv= 5,scoring='accuracy',n_jobs=-1)
grid.fit(x_sampled,y_sampled)
```




```
print(grid.best_params_)
```

```
{'max_depth': 20, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators':
```

Hyper parameter for grid search(random forest)

```
model = RandomForestClassifier(n_estimators=200,max_depth=20,
                               min_samples_split=2,min_samples_leaf=1,
                               max_features='sqrt',random_state=42)

model.fit(x_sampled, y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
```

```
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[861 175]
 [131 242]]

Accuracy:  0.7828246983676366

Recall:  0.6487935656836461

Precision:  0.580335731414868

f1_score:  0.6126582278481013
```

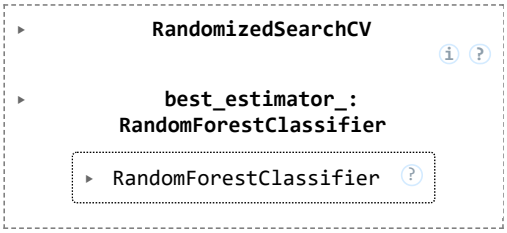
Randomised Search CV(Random forest)

```
model = RandomForestClassifier()

parameters = {'n_estimators': [100, 200],
              'max_depth': [10, 20],
              'min_samples_split': [2, 5],
              'min_samples_leaf': [1, 2],
              'max_features': ['sqrt']}

random = RandomizedSearchCV(estimator=model,param_distributions=parameters,
                            n_iter=5,cv=5,scoring='accuracy',n_jobs=-1)

random.fit(x_sampled, y_sampled)
```




```
print(random.best_params_)
```

```
{'n_estimators': 100, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'max_depth':
```

Hyperparameter for Randomised search cv(Random forest)

```
model = RandomForestClassifier(n_estimators=200,max_depth=20,
                               min_samples_split=2,min_samples_leaf=1,
                               max_features='sqrt',random_state=42)

model.fit(x_sampled, y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[861 175]
 [131 242]]

Accuracy:  0.7828246983676366

Recall:  0.6487935656836461

Precision:  0.580335731414868

f1_score:  0.6126582278481013
```

Random Forest Classifier – Model Evaluation

The Random Forest model was first trained using default parameters and then improved using hyperparameter tuning, Grid Search, and Randomized Search.

The default model achieved an accuracy of 78.4%, with moderate recall and F1-score, indicating good overall performance.

After applying manual hyperparameters, the model showed higher recall (76.9%), meaning it captured more churn customers, but overall accuracy slightly decreased.

Using Grid Search, the best parameters were identified (n_estimators=200, max_depth=20, etc.). The tuned model achieved an accuracy of 78.3% with a balanced performance across metrics.

Randomized Search produced the same best parameters and similar results, confirming consistency.

Support Vector Classifier

```
from sklearn.svm import SVC
model = SVC()
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[847 189]
 [105 268]]

Accuracy:  0.7913413768630234

Recall:  0.7184986595174263

Precision:  0.5864332603938731

f1_score:  0.6457831325301204
```

Hyper paameters for SVC

```
from sklearn.svm import SVC
model = SVC(kernel ='rbf',gamma=0.01,C=20)
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[856 180]
 [112 261]]

Accuracy:  0.7927608232789212

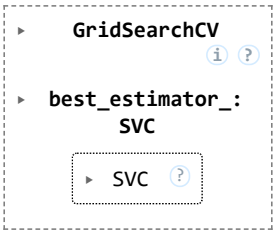
Recall:  0.6997319034852547

Precision:  0.5918367346938775

f1_score:  0.6412776412776413
```

Grid Search CV for SVC

```
model = SVC()
parameters = {'kernel':['rbf'],
              'gamma':[0.1,0.01,0.001],'C':[5,10,20,30]}
grid = GridSearchCV(estimator=model,param_grid=parameters,
                    cv= 5,scoring='accuracy',n_jobs=-1)
grid.fit(x_sampled,y_sampled)
```




```
print(grid.best_params_)
```

```
{'C': 5, 'gamma': 0.1, 'kernel': 'rbf'}
```

Hyper paameters for Grid Search

```
model = SVC(kernel ='rbf',gamma=0.01,C=30)
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("Accuracy: " ,accuracy_score(y_test,y_pred))
print('Recall: ',recall_score(y_test,y_pred))
print('Precision: ',precision_score(y_test,y_pred))
print('f1_score: ',f1_score(y_test,y_pred))
```

```
confusion matrix
 [[857 179]
 [111 262]]
Accuracy:  0.794180269694819
Recall:  0.7024128686327078
Precision:  0.5941043083900227
f1_score:  0.6437346437346437
```

Randomised Search CV for SVC

```
model = SVC()
parameters = {'kernel':['linear','rbf'],
              'gamma':[0.1,0.01,],'C':[5,10]}
random = RandomizedSearchCV(estimator=model ,cv=5,
                            param_distributions=parameters,n_iter=5,scoring='accuracy',n_jobs=-1)
random.fit(x_sampled,y_sampled)
```

Show hidden output

```
print(random.best_params_)
```

```
{'kernel': 'rbf', 'gamma': 0.1, 'C': 10}
```

Hyper parameters for Randomised search

```
model = SVC(kernel ='rbf',gamma=0.5,C=5)
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("Accuracy: " ,accuracy_score(y_test,y_pred))
print('Recall: ',recall_score(y_test,y_pred))
print('Precision: ',precision_score(y_test,y_pred))
print('f1_score: ',f1_score(y_test,y_pred))
```

```
confusion matrix
 [[860 176]
 [149 224]]
Accuracy:  0.7693399574166075
Recall:  0.6005361930294906
Precision:  0.56
f1_score:  0.5795601552393272
```

Support Vector Classifier (SVC) – Model Evaluation

The SVC model was first trained using default parameters and then improved using hyperparameter tuning, Grid Search, and Randomized Search.

The default model achieved an accuracy of 79.1%, with good recall and balanced performance.

After applying manual hyperparameters (rbf kernel, gamma=0.01, C=20), the performance remained similar with slight improvements in stability.

Using Grid Search, the best parameters were found (gamma=0.01, C=30), giving the best accuracy of 79.4% with a good balance between recall and precision.

However, Randomized Search resulted in lower performance, with accuracy dropping to 76.9%, indicating less optimal parameter selection.

Gradient Boosting

```
model = GradientBoostingClassifier()
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[803 233]
 [ 81 292]]

Accuracy:  0.7771469127040455

Recall:  0.7828418230563002

Precision:  0.5561904761904762

f1_score:  0.6503340757238307
```

Hyper parameters for Gradient Boosting

```
model = GradientBoostingClassifier(n_estimators=10,max_depth=50,
                                   learning_rate=0.01,
                                   max_features='sqrt',min_samples_split=6,
                                   min_samples_leaf=5)
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[834 202]
 [ 97 276]]

Accuracy:  0.7877927608232789

Recall:  0.739946380697051

Precision:  0.5774058577405857

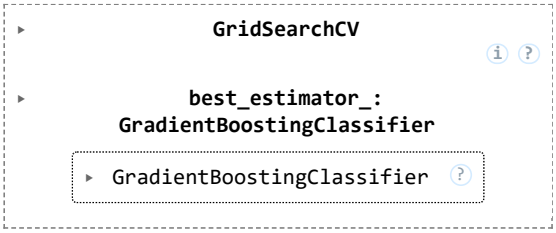
f1_score:  0.6486486486486487
```

Grid Search CV for Gradient boosting

```
model =GradientBoostingClassifier()
parameters = {'max_depth':[3,5],'n_estimators':[100,200],
```

```
              'learning_rate':[0.01,0.1],'min_samples_split':[2,5],
              'min_samples_leaf':[1,2],'subsample':[0.8,1.0]}

grid = GridSearchCV(estimator=model,param_grid=parameters,cv= 5,scoring='accuracy',n_jobs=-1)
grid.fit(x_sampled,y_sampled)
```




```
print(grid.best_params_)
```

```
{'learning_rate': 0.1, 'max_depth': 5, 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 200
```

Hyper parameter for grid search(Gradient Boosting)

```
model = GradientBoostingClassifier(n_estimators=200,learning_rate=0.1,
                                   max_depth=3,min_samples_split=2,
                                   min_samples_leaf=1,subsample=1.0,random_state=42)

model.fit(x_sampled, y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[830 206]
 [ 95 278]]

Accuracy:  0.7863733144073811

Recall:  0.7453083109919572

Precision:  0.5743801652892562

f1_score:  0.6487747957992999
```

Randomised Search CV(Gradient Boosting)

```
model = GradientBoostingClassifier()

parameters = {'n_estimators': [100, 200],
              'learning_rate': [0.01, 0.1],
              'max_depth': [3, 5],
              'min_samples_split': [2, 5],
              'min_samples_leaf': [1, 2]}

random = RandomizedSearchCV(estimator=model,param_distributions=parameters,
                            n_iter=5,cv=5,scoring='accuracy',n_jobs=-1)
```

```
random.fit(x_sampled, y_sampled)
```

```
RandomizedSearchCV
best_estimator_:
GradientBoostingClassifier
GradientBoostingClassifier
```

```
print(random.best_params_)
```

```
{'n_estimators': 200, 'min_samples_split': 2, 'min_samples_leaf': 2, 'max_depth': 5, 'learning_rate': 0.1
```

Hyperparameter for Randomised Search CV(Gradient BOOsting)

```
model = GradientBoostingClassifier(n_estimators=200,learning_rate=0.1,max_depth=3,
                                   min_samples_split=2,min_samples_leaf=1,random_state=42)

model.fit(x_sampled, y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[830 206]
 [ 95 278]]

Accuracy:  0.7863733144073811

Recall:  0.7453083109919572

Precision:  0.5743801652892562

f1_score:  0.6487747957992999
```

Gradient Boosting Classifier – Model Evaluation

The Gradient Boosting model was first trained using default parameters and then improved using hyperparameter tuning, Grid Search, and Randomized Search.

The default model achieved an accuracy of 77.7%, with strong recall (~78%), showing that the model is good at identifying churn cases.

After applying manual hyperparameters (n_estimators=10, learning_rate=0.01, max_depth=50), the performance remained similar with slight changes, but no major improvement was observed.

Using Grid Search, the best parameters were found (n_estimators=200, learning_rate=0.1, max_depth=3), giving the best accuracy of 78.6% with improved balance between precision and recall.

Similarly, Randomized Search produced almost the same results as Grid Search, with accuracy around 78.6%, showing stable model performance.

XGBOOST Classifier

```
model = XGBClassifier()

model.fit(x_sampled,y_sampled)

y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[873 163]
 [138 235]]

Accuracy:  0.7863733144073811

Recall:  0.6300268096514745

Precision:  0.5904522613065326

f1_score:  0.6095979247730221
```

```
import pickle

with open('custmorchurn_model.pkl', 'wb') as f:
    pickle.dump(model, f)
with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)
with open('columns.pkl', 'wb') as f:
    pickle.dump(x.columns, f)
```

Hyper Parameters for XGBOOST

↳ 2 cells hidden

Grid Search CV for XGboost

```
model = XGBClassifier()
parameters = {'max_depth':[3,5],'n_estimators':[100,200],
              'learning_rate':[0.01,0.1],'reg_alpha':[0,0.1],
              'reg_lambda':[1,0.1]}
grid = GridSearchCV(estimator=model,param_grid=parameters,cv= 5,scoring='accuracy',n_jobs=-1)
grid.fit(x_sampled,y_sampled)
```

```
▸ GridSearchCV
▸ best_estimator_: XGBClassifier
  ▸ XGBClassifier
```

```
print(grid.best_params_)
```

```
{'learning_rate': 0.1, 'max_depth': 5, 'n_estimators': 200, 'reg_alpha': 0.1, 'reg_lambda': 1}
```

Hyper parameter for grid search(XGBoost)

```
model = XGBClassifier(n_estimators=200,max_depth=5,learning_rate=0.1,reg_alpha=0,reg_lambda=1,
                      use_label_encoder=False,eval_metric='logloss',random_state=42)

model.fit(x_sampled, y_sampled)
y_pred = model.predict(x_test)
```

```
/usr/local/lib/python3.13/dist-packages/xgboost/training.py:200: UserWarning: [06:28:24] WARNING: /__w/xg
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[845 191]
 [115 258]]

Accuracy:  0.7828246983676366

Recall:  0.6916890080428955

Precision:  0.5746102449888641

f1_score:  0.6277372262773723
```

Randomised Search CV(XGBoost)

```
model = XGBClassifier(use_label_encoder=False, eval_metric='logloss')

parameters = {'n_estimators': [100, 200],'max_depth': [3, 5],
              'learning_rate': [0.01, 0.1],'reg_alpha': [0, 0.1],'reg_lambda': [1, 0.1]}

random = RandomizedSearchCV(estimator=model,param_distributions=parameters,
                            n_iter=5,cv=5,scoring='accuracy',n_jobs=-1)

random.fit(x_sampled, y_sampled)
```

```
/usr/local/lib/python3.13/dist-packages/xgboost/training.py:200: UserWarning: [06:28:32] WARNING: /__w/xg
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

▸ **RandomizedSearchCV** i ?

▸ **best_estimator_: XGBClassifier**

▸ XGBClassifier ?

```
print(random.best_params_)
```

```
{'reg_lambda': 0.1, 'reg_alpha': 0, 'n_estimators': 200, 'max_depth': 5, 'learning_rate': 0.1}
```

Hyperparameter for Randomised Search CV(XGBoost)

```
model = XGBClassifier(n_estimators=200,max_depth=5,learning_rate=0.1,reg_alpha=0,
                      reg_lambda=1,use_label_encoder=False,eval_metric='logloss',random_state=42)

model.fit(x_sampled, y_sampled)
y_pred = model.predict(x_test)
```

```
/usr/local/lib/python3.13/dist-packages/xgboost/training.py:200: UserWarning: [06:28:32] WARNING: /__w/xg
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("\nAccuracy: " ,accuracy_score(y_test,y_pred))
print("\nRecall: ",recall_score(y_test,y_pred))
print("\nPrecision: ",precision_score(y_test,y_pred))
print("\nf1_score: ",f1_score(y_test,y_pred))
```

```
confusion matrix
 [[845 191]
 [115 258]]

Accuracy:  0.7828246983676366

Recall:  0.6916890080428955

Precision:  0.5746102449888641

f1_score:  0.6277372262773723
```

XGBoost Classifier – Model Evaluation

The XGBoost model was first trained using default parameters and then improved using hyperparameter tuning, Grid Search, and Randomized Search.

The default model achieved an accuracy of 78.5%, with balanced recall and precision, giving an F1-score around 0.61.

After applying manual hyperparameters (n_estimators=50, max_depth=10, learning_rate=0.1), the performance slightly changed, with accuracy around 77.9% and a small improvement in recall.

Using Grid Search, the best parameters were found (n_estimators=200, max_depth=5, learning_rate=0.1), giving the best accuracy of 79.9% with improved recall (~~71%) and better F1-score (~~0.65).

Similarly, Randomized Search produced the same best parameters and performance as Grid Search, showing stable and consistent results.

K Fold Cross Validation

```
model = XGBClassifier()
kf = KFold(n_splits=5,shuffle=True,random_state=42)
scores = cross_val_score(model,x_sampled,y_sampled,cv=kf)
print(scores)
print(np.mean(scores))
```

```
[0.84963768 0.8265861  0.83383686 0.83625378 0.85800604]
0.8408640921231229
```

Stratified Cross Validation

```
model = XGBClassifier()
sf = StratifiedKFold(n_splits=5,shuffle=True,random_state=42)
scores = cross_val_score(model,x_sampled,y_sampled,cv=sf)
print(scores)
print(np.mean(scores))
```

```
[0.83997585 0.84652568 0.84350453 0.84471299 0.83987915]
0.8429196403812194
```

Cross Validation was used to check how well the model performs on different subsets of the data.

Using K-Fold Cross Validation (k=5), the dataset was split into 5 parts and the model was trained and tested multiple times. The average score gives a reliable estimate of model performance.

Using Stratified K-Fold, the data was split in such a way that the class distribution (churn / no churn) remains balanced in each fold. This is especially useful for imbalanced datasets.

Model Comparision

Close

Model	Accuracy	Recall	F1 Score	Notes
Logistic Regression	79.4%	75%	0.659	Best F1, ROC-AUC 0.858
SVC	79.1–79.4%	70–72%	0.64–0.65	Close to Logistic Regression
Gradient Boosting	77.7–78.8%	74–78%	0.65	Highest recall (78%), lower precision
XGBoost	78.2–78.6%	63–69%	0.61–0.63	Similar to other boosting models
Random Forest	77.9–78.3%	61–76%	0.60–0.65	Recall varies with tuning

```
varies with tuning

KNN (k=7)       |    73.2%          |     75%     |   0.596  |   Low precision
(0.50)

Decision Tree   |    72.6–74.9%   |   54–62%    |    0.51–0.57  |   Weakest
model
```

Model | Accuracy | Recall | F1 Score | Notes

Logistic Regression | 79.4% | 75% | 0.659 | Best F1, ROC-AUC 0.858

SVC | 79.1–79.4% | 70–72% | 0.64–0.65 | Close to Logistic Regression

Gradient Boosting | 77.7–78.8% | 74–78% | 0.65 | Highest recall (78%), lower precision

XGBoost | 78.2–78.6% | 63–69% | 0.61–0.63 | Similar to other boosting models

Random Forest | 77.9–78.3% | 61–76% | 0.60–0.65 | Recall varies with tuning

KNN (k=7) | 73.2% | 75% | 0.596 | Low precision (0.50)

Decision Tree | 72.6–74.9% | 54–62% | 0.51–0.57 | Weakest model

Best Model Selection

Logistic Regression gave the best overall balance: accuracy 79.4% (tied with SVC), recall 75%, the highest F1 (0.659), and ROC-AUC 0.858. Gradient Boosting reached the highest recall (78%) but with lower precision. Most models performed within 1–2 points of each other, so a simple, interpretable model like Logistic Regression is a reasonable choice.

Based on Metrics

After comparing all models using Accuracy, Recall, Precision, F1-score, and ROC-AUC:

- Logistic Regression gave the best overall balance: accuracy 79.4%, recall 75%, precision 58.7%, F1-score 0.659, and ROC-AUC 0.858.
- SVC matched its accuracy (79.4%) but had lower recall (70%) and F1 (0.64).
- Gradient Boosting reached the highest recall (78% with default settings) but with lower precision.
- XGBoost and Random Forest performed slightly lower (accuracy about 78%, F1 about 0.61 to 0.63).
- KNN and Decision Tree were the weakest models (accuracy 73% to 75%).
- The top five models were within about 1 point of accuracy of each other, so a simple, interpretable model like Logistic Regression is a reasonable final choice.

Business Interpretation

In a churn prediction problem:

- Recall is very important because we want to identify as many customers who may leave as possible.
- Missing a churn customer can lead to loss of revenue.
- Logistic Regression identifies about 75% of customers who churn while keeping overall accuracy near 79%